

Phase 1 —> 1A and 1B:

```
dc73181..23a0b72 main -> main
0 18s ~                                                                                                                                                                                                                                                                                                                                                                                                                                           
```

Phase 3 —> 3A and 3B:

```
0s ~ .. / PES1UG24CS288-pes-vcs ~ ? main
• ▶ ./pes init
Initialized empty PES repository in .pes/
0s ~ .. / PES1UG24CS288-pes-vcs ~ ? main
• ▶ /pes add file1.txt file2.txt
fish: Unknown command: /pes
0s ~ .. / PES1UG24CS288-pes-vcs ~ ? main
× ▶ ./pes add file1.txt file2.txt
0s ~ .. / PES1UG24CS288-pes-vcs ~ ? main
• ▶ ./pes status
Staged changes:
  staged:    file1.txt
  staged:    file2.txt

Unstaged changes:
(nothing to show)

Untracked files:
untracked:  .gitignore
untracked:  test_objects.c
untracked:  test_objects
untracked:  test_tree.c
untracked:  Makefile
untracked:  tree.c
untracked:  index.h
untracked:  commit.h
untracked:  README.md
untracked:  object.c
untracked:  pes.h
untracked:  test_sequence.sh
untracked:  tree.h
untracked:  index.c
untracked:  commit.c
untracked:  test_tree
untracked:  pes.c

0s ~ .. / PES1UG24CS288-pes-vcs ~ ? main
• ▶
```

```
0s ~ .. / PES1UG24CS288-pes-vcs ~ ? main
• ▶ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdec7e3f3a183d337639025de576db9ebb4 1776796092 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776796092 6 file2.txt
0s ~ .. / PES1UG24CS288-pes-vcs ~ ? main
• ▶ |
```

Phase 4 —> 4A, 4B and 4C:

17s ~ `../PES1UG24CS288-pes-vcs` ~ `main`

• `./pes log`

commit 464ef7d61f6b7a94f5c753d14ac610e3db8e722ccc0c65067488273ade23649f

Author: PES User <pes@localhost>

Date: 1776796870

Add farewell

commit 224cc219d4b880950789c842d74f482935a5285eeca11dc73504615a83dcf4d3

Author: PES User <pes@localhost>

Date: 1776796870

Add worldell

commit baf505e7e1a355a17dd5c048fc9f7653e8bd47a885c16c7bcb368cb4e0dbf6dd

Author: PES User <pes@localhost>

Date: 1776796870

Initial commit

0s ~ `../PES1UG24CS288-pes-vcs` ~ `main`

• `find .pes -type f | sort`

.pes/HEAD

.pes/index

.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43

.pes/objects/22/4cc219d4b880950789c842d74f482935a5285eeca11dc73504615a83dcf4d3

.pes/objects/46/4ef7d61f6b7a94f5c753d14ac610e3db8e722ccc0c65067488273ade23649f

.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39

.pes/objects/a6/882ee4afdc855ea4e39f4e0fc77d90e73a6f72deb383e11934310a10a030f6

.pes/objects/ba/f505e7e1a355a17dd5c048fc9f7653e8bd47a885c16c7bcb368cb4e0dbf6dd

.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442

.pes/objects/d8/f28203071e10a3bde605928368bc5b51871287a867d8d10382967099dde814

.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc

.pes/refs/heads/main

0s ~ `../PES1UG24CS288-pes-vcs` ~ `main`

• `cat .pes/refs/heads/main`

464ef7d61f6b7a94f5c753d14ac610e3db8e722ccc0c65067488273ade23649f

0s ~ `../PES1UG24CS288-pes-vcs` ~ `main`

• `cat .pes/HEAD`

ref: refs/heads/main

0s ~ `../PES1UG24CS288-pes-vcs` ~ `main`

•

Phase 5 and 6:

Q5.1) How to implement ``pes checkout <branch>``

Files to change in ``.pes/``:

1. Read ``.pes/refs/heads/<branch>`` to get target commit hash.
2. Update ``.pes/HEAD`` to ``ref: refs/heads/<branch>`` (normal branch checkout).
3. Rebuild ``.pes/index`` from the target commit's root tree.

Working directory actions:

1. Read target commit object, then its root tree.
2. Materialize target tree into files/directories:
 - create/update files that exist in target tree,
 - set executable bit from tree mode,
 - remove tracked files that exist in current branch but not in target branch.

Why complex:

- Must protect uncommitted work.
- Must detect conflicts between local modifications and files that will change on checkout.
- Must handle recursive trees, deletions, mode changes, and partial failures atomically/safely.

Q5.2) Detect dirty working directory conflict using index + object store

Idea: compare three states per tracked path:

- Current index entry (what user staged/last known metadata),
- Working directory content (current file on disk),
- Target branch content (blob hash from target commit tree).

Steps:

1. Resolve target commit and expand its tree into a map: ``path -> target_blob_hash``.
2. For each path tracked in current index (or present in target map):
 - Determine if working file is dirty relative to index:
 - fast check: ``mtime`/size mismatch,`
 - strict check: hash working file and compare with index hash.
 - Determine if checkout would modify that path:
 - target hash differs from current tracked hash, or file would be deleted/created.
3. Conflict rule:
 - if file is dirty AND checkout would modify same path => refuse checkout.

This prevents overwriting user changes with branch content.

Q5.3) Detached HEAD behavior and recovery

Detached HEAD means ``.pes/HEAD`` stores a commit hash directly, not a branch ref.

If commits are made in detached state:

- New commits are created normally.
- ``.HEAD`` moves to the new commit hash.
- No branch name advances, so those commits are easy to "lose" from normal branch history.

Recovery:

1. Identify detached commit hash (from `pes log`, `HEAD`, or reflog-like history if available).
2. Create/move a branch ref to it, e.g., write hash into `.pes/refs/heads/<new-branch>`.
3. Point `HEAD` back to that branch (`ref: refs/heads/<new-branch>`).

After that, commits are reachable and no longer dangling.

Q6.1) Algorithm to find and delete unreachable objects

Goal: delete objects not reachable from any live reference.

Mark-and-sweep approach:

1. Start roots:
 - all branch heads in `.pes/refs/heads/\*`
 - (optionally) detached `HEAD` if it directly contains a hash
2. Mark phase (graph traversal):
 - Use a stack/queue of object IDs to visit.
 - For each object:
 - if commit: mark it, enqueue its `tree`, enqueue `parent` (if present)
 - if tree: mark it, parse entries, enqueue child tree/blob hashes
 - if blob: mark it only (leaf)
3. Sweep phase:
 - scan all files under `.pes/objects/\*\*`
 - convert each filename to object hash
 - if hash not in marked set => delete object file

Best data structure for reachable set:

- Hash set (`unordered\_set` style) keyed by 32-byte hash (or 64-char hex).
- O(1) average insert/lookup during traversal and sweep.

Scale estimate (100,000 commits, 50 branches):

- If branches mostly share history, unique commits visited are near 100,000 (not 5,000,000).
- Objects visited are roughly:
 - ~100,000 commit objects
 - up to ~100,000 root tree objects (plus subtrees depending on project depth)
 - blobs referenced by those trees (often many shared)
- So total visits are on the order of repository object count (commits + trees + blobs), commonly hundreds of thousands to low millions.

Q6.2) Why concurrent GC is dangerous and how Git avoids races

Race example:

1. Commit process writes new blob/tree objects, but has not yet updated branch ref.
2. Concurrent GC starts from current refs and marks reachable objects.
3. New objects are not yet referenced by any ref, so GC treats them as unreachable.
4. GC deletes them.

5. Commit process finally writes commit/ref pointing to now-deleted objects => repository corruption/broken history.

Why dangerous:

- Reachability is time-dependent; between "object written" and "ref updated" there is a vulnerable window.

How real Git avoids this (conceptually):

- Uses locking/coordination around ref updates and maintenance tasks.
- Uses temporary object protection and grace periods (reflogs, recent-object heuristics, prune with age threshold).
- Runs aggressive prune only when objects are old enough and unlikely to be part of in-flight writes.
- Performs atomic ref updates so roots change consistently.

Result: objects created by ongoing operations are not collected prematurely.